

90 MINUTES • AI TEAM

AI Engineering Starter Kit

How we divide engineering labor, and which commands actually do that work.

AGENDA

How the hour is split

Block	Minutes	Focus
Method	~45	Labor, ownership, Explore, pipeline, evidence
Tooling	~30	./ask, workstreams, skills, Cursor binding
Demo	~5	Where the facilitator script lives, and what it proves
Discussion	~10	What we would try on a real product repo

WHAT THIS IS

A protocol you copy into a software repo

Portable repository protocol for delegating engineering labor to AI agents while the human keeps ownership of intent, policy, and acceptance.

Cursor is the first-class runtime binding. The protocol stays copyable into ordinary software repos.

The kit is Guide + Build Spec + stage contracts + small scripts + pinned Community Skills. It is not a multi-agent runtime product.

The Guide's central question

Which engineering labor should be delegated, which judgment should remain human, what authority does each actor have, what evidence is required, and how can the resulting work remain traceable?

Engineering labor is several classes of work

Execution: write code, edit files, run commands

Cognitive: compare alternatives, build a plan

Judgment: decide what is appropriate under uncertainty

Verification: check a claim against evidence

Coordination: pass structured outputs and keep state

Governance: who may decide or act, and when to escalate

AI can perform some amount of all of these. The practical question is where delegation stays trustworthy and economically sensible.

DO NOT COLLAPSE THESE

Five questions that stay separate

Word	Asks
Labor	Who performs the work?
Judgment	Who decides what is correct, appropriate, or preferable?
Capability	What can this actor technically do?
Authority	What is this actor permitted to decide or enact?
Accountability	Who is responsible for the consequences?

A database migration

Labor → AI prepares migration

Judgment → AI analyzes expected impact

Capability → tool can execute migration

Authority → human approval required for production

Accountability → human/team

Delegating labor does not automatically delegate ownership.

Own the system that turns intent into accepted software

As implementation labor moves to agents, code-level ownership can shrink. Engineering ownership does not have to disappear with it.

You remain responsible for What/Why, constraints, acceptance criteria, architecture boundaries, delegation policy, escalation rules, evidence policy, reusable methods, decision memory, and provenance.

You can reuse a grilling skill or a spec method without having authored it. You still own the decision to rely on it here, and the consequences.

This is still an engineering job

A Product Owner may primarily own What, Why, and business priority.

An engineering-system owner also governs technical constraints, architecture, acceptance evidence, verification policy, delegation boundaries, escalation, method composition, decision memory, and provenance of results.

You can know less about a given diff and more about the system that produced and accepted it. That changes the level of ownership. It does not automatically make the role superior.

Explore when the destination is foggy

00 Explore charts decisions, research, and cheap prototypes until What/Why can be owned. It produces decisions and clarity. It does not produce accepted product software by itself.

Skip Explore when the destination is already sharp enough for Intent. That is a real skip: no explore-map.md.

Do not stuff R&D into Implement.

The Engineering Pipeline is 01-10

01 Grill → 02 Spec → 03 Spec Challenge

04 Spec Change (interrupt)

05 Plan → 06 Implement → 07 Review → 08 Refactor

09 Verify → 10 Accept

Each next stage stands on the previous artifact. After one "defaults OK," you skip extra blessings, not the documents. Accept is the second confirm.

/off-path (or "just code") leaves the path for this chat only. A new chat starts on-path.

Grill reduces ambiguity. Challenge asks if the spec is the right problem.

A Grill agent that only restates "make cancellation fast" has not done the job. Alternatives, tradeoffs, and failure modes have to be visible before "all ok."

A specification challenge asks whether the spec is a faithful representation of the intended problem. Coherent is not the same as correct.

An implementation problem may generate a spec-change proposal. The implementer must not silently edit the accepted spec to make the code easier.

Verification is claim-dependent

Do not make the implementing agent the sole judge of its own correctness. For important claims, prefer evidence whose failure mode differs from the agent that wrote the code.

A type check, a behavioral test, an architectural check, a benchmark, and a formal proof answer different claims. None of them is universally stronger.

Acceptance can be delegated as labor. Ultimate authority need not be. The gap between lots of autonomous changes and few strong evidence packages is acceptance debt.

Store engineering state, not every token

Durable state: intent and spec, policy and skill versions, the code commit, the verification result, the acceptance decision.

Every material experiment or benchmark should identify the exact commit that produced it.

Git invariants: start from a clean tree; one plan per agent/<work-id> branch; do not pile a new task on another task's uncommitted work; commit meaningful states; record result → exact commit.

Delegate the bounded work. Keep the consequential calls.

Delegate labor aggressively where the work is bounded, reversible, and verifiable; preserve human judgment and authority where intent, trade-offs, risk, or irreversible consequences dominate; and move critical constraints from probabilistic instructions into deterministic enforcement whenever practical.

Repeated failures should change the system (a check, a rule, a better spec template), not only the next prompt.

What lands in a product repo

Kit package _ask/ protocol, scripts, kit tests, kit-author docs

Adapter ask, AGENTS.md, .cursor/

Product state specs/, work/

Your app docs/, scripts/, tests/, src/ stay yours

./ask is the only root command. It execs _ask/scripts/. Agents must not run ./ask setup.

Three verbs, three objects

Command	Object	Who
<code>./ask install <repo></code>	Kit files: <code>_ask/</code> , <code>ask</code> , <code>AGENTS.md</code> , <code>.cursor/</code>	Agent or human. Target must already be a git repo. Leaves <code>docs/</code> , <code>scripts/</code> , <code>tests/</code> , <code>src/</code> alone.
<code>./ask prepare</code>	Pinned Community Skills from <code>_ask/skills/manifest.yaml</code>	Agent or human. Refuses revision: latest. Bodies land in <code>.agents/skills/</code> (gitignored).
<code>./ask setup</code>	Your tracker + Cursor MCP stubs	You, on a TTY. Writes gitignored <code>.ask.env</code> . Agents must not run it.

`./ask sync` is a fourth verb: regenerate `.cursor/skills` and `.cursor/commands` from `_ask/agents/`. It does not copy the kit and it does not fetch skills.

Which list you are on

Repo already has the kit (this one, or after overlay):

1. `./ask prepare`
2. `./ask sync`
3. Work. Skip install. Skip setup unless you want the wizard.

Adding the kit to a different app repo:

1. `./ask install ../my-app` or `askit first run`
2. In that repo: prepare, then sync
3. You run setup only for tracker/MCP

One plan, one branch, commits as you go

`./ask start-work <work-id>` creates `agent/<work-id>` and seeds `work/<work-id>/`. It refuses a dirty tree. It does not seed `explore-map.md`.

On that branch, commit each meaningful step. Do not wait to be asked. Push, force-push, amend of pushed commits, and merge to main still need a human.

`./ask status` lists live `agent/*` and archived `work/*` on the default branch. The current checkout is not the board.

COMMAND MAP

What you will type after the kit is in

Command	Job
<code>./ask check-clean</code>	Refuse if the tree is dirty
<code>./ask start-work</code>	Branch + seed artifacts
<code>./ask check-workstream</code>	Preconditions before implement
<code>./ask verify</code>	Run checks; print commit SHA
<code>./ask record-result</code>	Workstream provenance (needs <code>--commit-sha</code>)
<code>./ask record-run</code>	Experiment provenance; SHA must be HEAD; tree must be clean
<code>./ask upgrade --version</code>	Refresh kit-owned files from an explicit tag or SHA

Pins, prepare, thin binding

Community Skills are declared in `_ask/skills/manifest.yaml` with an explicit revision. Discovery can start at `skills.sh`. The project owns the pin.

Capability is not authority. A rule is not enforcement. A skill is not a guarantee. Move critical constraints into hooks, scripts, and CI when you can.

Cursor rules point at protocol. `./ask sync` writes `.cursor/skills` and `.cursor/commands` from `_ask/agents/`. Do not treat generated projections as the source of truth.

A facilitator script exists if you want the lab later

Script: `_ask/docs/demo/end-to-end-plan.md`. About 45-90 minutes for Path A. Vehicle: SHA-bound moons MLP runs.

What it proves: durable intent and spec; clean tree + agent/<work-id>; `./ask record-run` so each metric cites path + git SHA; spec changes go through Spec Change; skills come from a pin + prepare.

After that lab you should be able to answer: which experiment, which metric, which SHA, which artifact path, who accepted the workstream. The registry is the scoreboard.

A conservative first pass

Read root AGENTS.md. Run ./ask prepare and ./ask sync in a repo that already has the kit.

One workstream: clean tree, dedicated branch, grill What/Why, write the artifacts, commit as you go, ./ask verify, record the result against HEAD, accept with a SHA.

The kit does not assume one model is enough, that several agents are independent reviewers, that a long rule file is formal governance, or that AI may silently redefine requirements.

DISCUSSION

Where would you spend human judgment first?

Intent, irreversible changes, and evidence policy are the usual starting points. A silent spec rewrite from last month is a concrete one.